

3: k -Nearest Neighbors, Curse of Dimensionality*Date: January 22, 2026*

1 Assumptions about Data: a Motivating Example

Consider the following simple dataset:

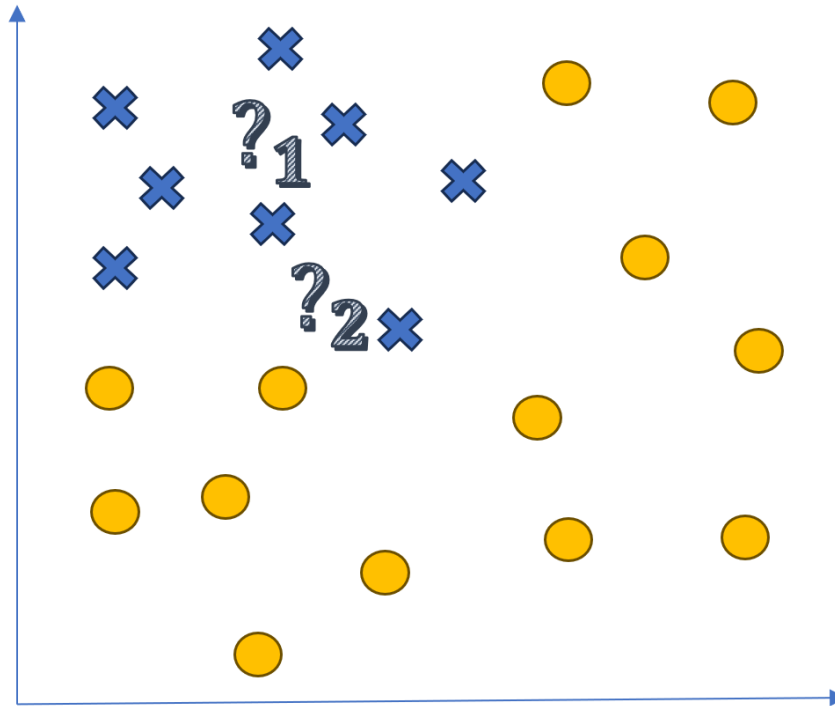


Figure 1: A very simple dataset, where each axis of the plot corresponds to an input feature. Each data point has either a label of $y = 1$ (the blue crosses) or $y = -1$ (the yellow circles).

If you were asked to label the point $?_1$, you'd probably tell me that it's a cross (e.g., $y = 1$). Similarly, you might tell me that $?_2$ is a circle although this is probably less clear.

It's important to realize at this stage that what you've done is made an *assumption* about the data. Before you read on, it's worth trying to distill down precisely what assumption you made about the data that would lead you to those two labeling.

If you're anything like most people who get asked about this, your intuition can probably be distilled down to something like:

$?_1$ is a cross because it's closer to other crosses than it is to circles, and $?_2$ is probably a cross, but might not be because it's pretty close to a circle.

The idea behind k-Nearest neighbors is just to make an algorithm out of this assumption!

2 k-Nearest Neighbors

As usual in machine learning, we assume we are given a training dataset $\mathcal{D} = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)\}$.

In k-Nearest neighbors (k-NN), the learning procedure consists of: nothing! The model is our data.

Given a test input $\mathbf{x}$, We define the k nearest neighbors of $\mathbf{x}$, $N_{\mathbf{x}} \subseteq \mathcal{D}$ as the set of k points so that any point not in $N_{\mathbf{x}}$ is further from $\mathbf{x}$ than all points in $N_{\mathbf{x}}$. That is:

$$\forall \mathbf{z} \in \mathcal{D} \setminus N_{\mathbf{x}}, \mathbf{z}' \in N_{\mathbf{x}} \quad \text{dist}(\mathbf{x}, \mathbf{z}) \geq \text{dist}(\mathbf{x}, \mathbf{z}') \quad (1)$$

The k-Nearest neighbors classifier takes the majority vote of the labels in $N_{\mathbf{x}}$:

$$h(\mathbf{x}) = \text{mode}(\{y \mid (\mathbf{x}, y) \in N_{\mathbf{x}}\}) \quad (2)$$

Distance. It's pretty clear that the above classifier relies pretty heavily on a good notion of distance. If the distance metric you use doesn't reliably capture notions of "farness" and "closeness", your k-NN classifier won't be very good! The Minkowski distances are some of the most popular when $\mathbf{x} \in \mathbb{R}^d$, and are defined by:

$$\text{dist}(\mathbf{x}, \mathbf{z}) = \left(\sum_{i=1}^d |[\mathbf{x}]_i - [\mathbf{z}]_i|^p \right)^{1/p} \quad (3)$$

This looks annoying, but think about what happens in the special cases of $p = 1$, $p = 2$, and $p = 3$. These are all "distances" you may have encountered in the past!

NCD. A pretty cool distance that showed up in a recent paper¹ at ACL 2023 is the normalized compression distance. The normalized compression distance is defined when the inputs $\mathbf{x}$ are text documents rather than real valued vectors. In other words, think of each x_i as a sequence of English words of possibly different lengths.

Denote by $\text{gzlen}(x)$ the length of x after being compressed by the gzip compression tool. Denote by $x_i \oplus x_j$ the string concatenation of x_i and x_j . For example `'Machine learning'  $\oplus$  'is fun' = 'Machine learning is fun'`. The normalized compression distance is given by:

$$\text{NCD}(\mathbf{x}, \mathbf{z}) = \frac{\text{gzlen}(\mathbf{x} \oplus \mathbf{z}) - \min\{\text{gzlen}(\mathbf{x}), \text{gzlen}(\mathbf{z})\}}{\max\{\text{gzlen}(\mathbf{x}), \text{gzlen}(\mathbf{z})\}} \quad (4)$$

The key term doing a lot of the work intuition wise is $\text{gzlen}(\mathbf{x} \oplus \mathbf{z})$. The idea is that, if $\mathbf{x} = \mathbf{z}$ then compression works extremely well because $\mathbf{x} \oplus \mathbf{z}$ is just the same string duplicated! Here's a simple implementation of this for sentiment analysis: <https://github.com/Sentdex/Simple-kNN-Gzip>.

¹<https://aclanthology.org/2023.findings-acl.426.pdf>

3 When does k-NN Fail? Case 1

k-NN is actually a pretty powerful model overall. Using the NCD distance metric, k-NN does almost as well as modern transformers – the T in GPT – sometimes! The original paper reports better performance than a popular transformer model, but actually had an error in it that overreported accuracy by about 5%.

The most obvious case where k-NN will not work is when *the distance metric you've chosen is very poor for the data you have*. A really simple example of this is natural images. In our first lecture, we discussed mapping the RGB values of pixels in an image to (really long) vectors $\mathbf{x}$. If I were to use something like Euclidean distance (the Minkowski distance with $p = 2$), this works *notoriously* poorly for images.

To see why this is the case, consider the following two images (Figure 2):



Figure 2: Two of the same image. Or are they?

At first glance, these two images seem to be identical. But actually, they have **gigantic** Euclidean distances if you reduce them down to vectors. The reason for this is simple: in the rightmost image, I shifted all of the pixels one to the right. Thus, all the boundaries of the blades of grass and stripes of the cat don't quite match up, and there are **tons** of pixels that are very dissimilar!

This sort of intuition is *precisely* what results in some of the success of deep learning over other learning paradigms: the assumptions that many classic machine learning algorithms make just work very poorly on image data.

4 When does k-NN Fail? Case 2

It's time to talk about **the curse of dimensionality**. Suppose my training data is drawn uniformly at random from the unit cube $[0, 1]^d$. Consider a test point $\mathbf{x}$, and suppose the smallest hypercube that contains $N_{\mathbf{x}}$ has side length ℓ . Then, $\ell^d \approx \frac{k}{n}$, and $\ell = \left(\frac{k}{n}\right)^{\frac{1}{d}}$. With $k = 1$ and $n = 100$, in two dimensions $\ell \approx 0.1$. When $d = 10$, $\ell \approx 0.63$. When $d = 100$, $\ell \approx 0.95$. When $d = 1000$, $\ell \approx 0.9954$.

What does this mean? Uh oh, your k nearest neighbors are *on the other side of the space!*. Another way to think about this is the following. Imagine you have a test point $\mathbf{x} = [1, 1, \dots, 1]$. If a training point $\mathbf{z}$ is uniformly randomly placed in the unit cube $[0, 1]^d$, what is the probability that **none** of the entries in $\mathbf{z}$ are smaller than ϵ ? Well, the probability of being greater than ϵ in a single uniform

component is $1 - \epsilon$, and this needs to be true for all d components, so $(1 - \epsilon)^d$. When $\epsilon = 0.05$ and $d = 1000$, there is only a 5×10^{-23} chance! Thus, **every** training point is on the opposite end of the box from $\mathbf{x}$ in at least one of the input dimensions.